BONUS: RegEx Matching Lab (CodeGrade)

(https://github.com/learn-co-curriculum/python-p3-regex-lab) (https://github.com/learn-co-curriculum/python-p3-regex-lab/issues/new)

Learning Goals

- Validate that strings match specific patterns using regular expressions.
- Search for strings that match specific patterns using regular expressions.

Key Vocab

- **Regular Expression**: a sequence of characters used to search for a pattern inside of a string.
- **Pattern**: a description of sequences of characters that share certain traits with one another. Sequences do not need to be the same length or share any common characters to pattern match. Also called a **filter**.

Instructions

This is a **test-driven lab**. Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment. Then run `pytest -x` to run your tests. Use these instructions and `pytest` 's error messages to complete your work in `regex.py` in the `lib` folder.

This lab is meant to give you experience with writing patterns for regular expressions and using the `re` module to check your pattern against different strings. Use **regex101** (https://regex101.com/) to test your pattern as you work through the different strings.

- Run `pytest -x` to execute the first test. This will check your RegEx against the string `"It's such a lovely day today."`
- Once you've compiled a regular expression that matches this first string, run `pytest -x` again to check against the next string in line.

- Your RegEx should match **all** of the strings in the test file!
- The last test uses the `findall()` method to search for these three strings in a list. You can check for this list in `testing/regex_test.py`.
  - ***If you hacked your way through the first three with `r".*"`, you'll get caught here!***

Once all of your tests are passing, commit and push your work using `git` to submit.

Hints

Regular expressions can be confusing and frustrating. You'll only get used to them with practice. That being said, there are some tools and tricks that will help you our along the way:

- `\w` is shorthand for any word character: `[A-Za-z0-9_]`. `\W` is shorthand for anything outside of that range.
- Similarly to `\w`, `\S` will match any non-whitespace character- though it includes characters not found in words as well. `\s` matches any single whitespace character.
- Getting quotes, backslashes, and other operational characters into your regex requires that you lead with an escape character, the backslash (`\`).
- Ignoring a specific set of characters (say, 'c', 'h', 'a', 'r', 's') can be accomplished with square brackets and carrots (`^`): [^chars].
- Matching one pattern *or* another with the same regex can be accomplished with the `|` operator: `r'hello|goodbye'`.
- **regex101 (https://regex101.com/)** provides a "Quick Reference" section in the bottom right corner that is very helpful for more tips and tricks along the way.

Resources

- **re - Regular expression operations - Python (https://docs.python.org/3/library/re.html)**
- **regex101 (https://regex101.com/)**
- **Python Regular Expressions - Google for Education (https://developers.google.com/edu/python/regular-expressions)**

This tool needs to be loaded in a new browser window

Load BONUS: RegEx Matching Lab (CodeGrade) in a new window